Компиляция статической маршрутизации с нелинейностью до агрегации

Аннотация. В многослойном перцептроне можно заменить линейное преобразование с последующей активацией на среднее нелинейных откликов отдельных входных координат. В работе исследуется такой слой со статическими обучаемыми вероятностями и одним скалярным весом и сдвигом на нейрон. На CIFAR-10 модель с факторизованными логитами вероятностей имеет 36 426 параметров и достигает точности 50,58% против 50,81% у полносвязного контроля с 570 570 параметрами. На MNIST и Fashion-MNIST полносвязные модели в соответствующих сериях точнее. Компактная параметризация сокращает размер сохранённых весов, однако при развёртывании в плотную матрицу вероятностей возникает дополнительный кеш. Для уменьшения этого кеша предложен компилятор, который хранит отдельные маршруты и массы отброшенных входов, а кусочно-линейный отклик хвоста вычисляет по общим префиксным статистикам для индивидуальных порогов нейронов. После калибровки на 1536 обучающих изображениях потеря точности относительно исходной маршрутизирующей модели составляет 0,35, 0,14 и 0,27 процентного пункта на CIFAR-10, Fashion-MNIST и CIFAR-100 соответственно. На центральном процессоре Apple M3 Pro в однопоточном режиме, при обработке одного изображения и хранении вероятностей в 32-битном формате, скомпилированный классификатор работает в 2,81 раза быстрее точного кешированного вычисления и использует в 10,3 раза меньше памяти для явных массивов и рабочих буферов. Оптимизированный полносвязный контроль сохраняет преимущество по задержке. Результаты характеризуют условия, при которых сжатое исполнение выгодно для уже обученной маршрутизирующей модели.

Ключевые слова. Статическая маршрутизация, многослойный перцептрон, кусочно-линейная активация, сжатие нейронных сетей, префиксные суммы, задержка вывода.

1. Введение

Полносвязный скрытый слой сначала смешивает входные координаты, затем применяет скалярную нелинейность. Рассмотрим другую архитектуру. Нейрон назначает входным координатам вероятности, преобразует значение каждой координаты одной и той же скалярной функцией и суммирует отклики с этими вероятностями. Распределение обучается вместе с функцией и остаётся фиксированным для всех объектов. Такая модель может непосредственно использовать свойства распределения значений, которые исчезают при линейном усреднении до активации.

Статические факторизованные логиты встречаются в Synthesizer [1]. Суммы одномерных функций характерны для сетей Колмогорова--Арнольда [2], а индивидуальные пороги выходов рассматриваются в TruKAN [3]. В исследуемом слое для всех входных координат одного выхода используется одна скалярная функция. Это ограничение позволяет разным выходам переиспользовать статистики одних и тех же групп входов. Первую часть работы составляет проверка качества такого слоя относительно многослойного перцептрона с полносвязными слоями (Dense MLP).

Факторизация логитов уменьшает число обучаемых параметров, а построчный softmax обычно разрушает факторизацию самих вероятностей. При точном выводе можно развернуть вероятности в кеш, размер которого сопоставим с матрицами Dense MLP. Дополнительная трудность состоит в том, что отклик каждого входа зависит от параметров выходного нейрона. Готовое среднее входов нельзя без потерь подставить в его активацию, если входы лежат по разные стороны порога.

Во второй части предлагается представление с явными маршрутами и остаточной массой по группам. Компилятор использует разреженную поправку и общие статистики групп,

вычисляя отклики для индивидуальных порогов. Под точностью исполнения понимается совпадение с математической формой сжатого слоя с точностью арифметики. Замена исходной матрицы вероятностей сжатой оценивается отдельно.

Эксперимент проверяет три вопроса. Насколько качество архитектуры меняется при применении нелинейности до суммы и при факторизации логитов? Какой вклад в сохранение качества вносят пространственные группы и точная сумма активаций по хвосту? Как меняются точность, размер представления и задержка всего классификатора после компиляции? Последний вопрос рассматривается как для одной и той же маршрутизирующей сети, так и относительно независимых полносвязных контролей.

2. Связанные работы

Factorized Random Synthesizer строит статическую матрицу внимания как softmax произведения обучаемых низкоранговых факторов [1]. В этой работе используется та же идея для параметризации вероятностей. Здесь значение на ребре зависит от выходного нейрона через его скалярную функцию. Сети Колмогорова--Арнольда задают послойную агрегацию одномерных функций на рёбрах [2]. TruKAN использует усечённые степенные базисы, включая вариант с порогами, общими внутри одного выхода [3]. Рассматриваемый слой является ограниченным семейством таких аддитивных преобразований, где коэффициенты рёбер неотрицательны и нормированы по строке.

SparQ восстанавливает часть вклада невыбранных позиций через среднее значений [4], а OpenAttention использует групповые прокси хвоста [5]. В статической постановке массы маршрутов известны заранее. Индивидуальный порог каждого выхода требует сохранить в вычислении сумму активаций, а не только среднее входных значений. Для вычисления применимы известные идеи гистограмм и быстрых префиксных сумм [6, 7]. Вклад работы состоит в их сочетании с формой обученного слоя, построении полного исполнителя и абляциях, отделяющих эффект группировки от эффекта нелинейного отклика.

Восстановление выходов слоёв на небольшой калибровочной выборке используется в методах разреживания, например в SparseGPT [8]. В нашей подгонке нормировка и неотрицательность масс следуют из вероятностного представления. Структурированные линейные операторы, в частности Butterfly [9], показывают, что компактность параметров и скорость могут достигаться и в полносвязных моделях. Поэтому сравнение с исходным кешем проверяет эффективность компиляции фиксированной сети, а сравнение с факторизованным и квантованным Dense проверяет альтернативный выбор модели при том же бюджете хранения.

3. Постановка задачи

Пусть $\{(x^{(s)}, c^{(s)})\}_{s=1}^{n}$ — выборка размеченных изображений, $x^{(s)} \in \mathbb{R}^{d_0}$ и $c^{(s)} \in \{1, \dots, C\}$. Вектор получается разворачиванием нормализованного изображения. Выборка разделяется на обучение, валидацию и тест. Размеры и предобработка приведены в разделе 5.1. Сеть F_θ с L скрытыми слоями возвращает логиты $z = F_\theta(x)$. Параметры обучаются минимизацией средней кросс-энтропии

$$\mathrm{CE}(F_\theta; D) = \frac{1}{|D|} \sum_{(x,c)\in D} \left(\log \sum_{k=1}^{C} e^{F_\theta(x)_k} - F_\theta(x)_c \right). \tag{1}$$

Доля верных ответов Acc определяется по $\arg\max_k z_k$. Для каждого запуска выбирается самый ранний чекпойнт с максимальной валидационной точностью среди сохранённых проверок. Тест служит для итогового измерения.

Задача компиляции задаётся для уже обученной сети. Даны вероятности маршрутов P и параметры активаций. Требуется построить представление Q и алгоритм вывода F_Q, используя заданное число явных маршрутов на строку и общие группы входов. Критерии сравнения — разность $\mathrm{Acc}(F_Q) - \mathrm{Acc}(F_P)$, изменение (1), число байтов хранимых массивов и рабочих буферов и задержка полного вывода на центральном процессоре. Конфигурация сжатия выбирается по валидации. При подгонке используются только входы обучающего разбиения и отклики исходной модели.

4. Архитектура статической маршрутизации

Для слоя с N входами и M выходами зададим

$$h_m(x) = \sum_{i=1}^{N} p_{mi}\, g(w_m x_i + b_m), \qquad g(u) = \tfrac{1}{2}u + \tfrac{3}{2}\,[u]_+\,, \quad [u]_+ = \max(u, 0). \tag{2}$$

Здесь $w_m, b_m \in \mathbb{R}$ общие для всех i одного выхода, а $p_{mi} \geq 0$, $\sum_i p_{mi} = 1$. Выражение (2) равно условному ожиданию $g(w_m x_{I_m} + b_m)$ при $I_m \sim p_{m\cdot}$. На следующий слой поступает вычисленный вектор h. Послойная подстановка ожиданий задаёт детерминированную сеть. Если случайные маршруты сэмплировать во всех слоях, применение следующей нелинейности к случайному вектору в общем случае даёт другое ожидание.

В полном варианте P — построчный softmax обучаемой матрицы логитов. В компактном варианте

$$P = \mathrm{softmax}_{\mathrm{row}}(UV^{\mathsf{T}}/\sqrt{r}), \qquad U \in \mathbb{R}^{M\times r}, \quad V \in \mathbb{R}^{N\times r}. \tag{3}$$

Слой содержит $r(N + M) + 2M$ обучаемых параметров вместо $MN + 2M$. Матрица P после softmax может иметь ранг выше r. Для четырёх скрытых слоёв ширины 160, входа из 3072 координат, линейной шапки на 10 классов и $r = 8$ это даёт 36 426 параметров. Полный Dense MLP той же ширины имеет 570 570 параметров.

Положение активации. Контроль с активацией после суммы использует то же P и вычисляет $g(w_m \sum_i p_{mi} x_i + b_m)$. Пусть $Z = w_m x_{I_m} + b_m$. Тогда

$$\mathbb{E}g(Z) - g(\mathbb{E}Z) = \tfrac{3}{2}\left(\mathbb{E}\,[Z]_+ - [\mathbb{E}Z]_+\right) = \tfrac{3}{2}\min\!\left(\mathbb{E}\,[Z]_+\,, \mathbb{E}\,[-Z]_+\right). \tag{4}$$

Второе равенство следует из $Z = [Z]_+ - [-Z]_+$. Разность равна нулю, когда вся ненулевая масса лежит на одной ветви. При пересечении порога она зависит от величин откликов по обе стороны. Эту разность далее измеряем в абляции хвоста при одинаковых вероятностях.

Пример выразительности. Два выхода с равномерным распределением, $w = 1$ и $w = -1$, $b = 0$ вычисляют среднее $|x_i|$ после линейного объединения, поскольку $g(t) + g(-t) = \frac{3}{2}|t|$. Рассмотрим класс моделей вида $u(Ax)$, где A — линейная проекция ранга $r < N$, а u — произвольная функция. На кубе $[-1, 1]^N$ ошибка равномерного приближения среднего модулей в этом классе не меньше $(N - r)/(2N)$. Действительно, в вершине многогранника $\ker A \cap [-1, 1]^N$ зафиксировано на границе не меньше $N - r$ координат. Эта точка и нуль имеют одинаковый образ после A, а их целевые значения отличаются минимум на $(N-r)/N$. Пример различает два порядка операций при сжимающем первом линейном отображении. Он характеризует конкретное семейство функций. Общая эквивалентность классов архитектур и качество их оптимизации требуют других условий.

5. Классификация и сравнение с Dense

5.1. Данные и протокол

MNIST [10] и Fashion-MNIST [11] содержат изображения 28×28 одного канала и десять классов. В обоих случаях для обучения, валидации и теста выделено 55 000, 5000 и 10 000 изображений. Для разбиения MNIST использована перестановка с начальным состоянием генератора 20261110. Интенсивности MNIST делятся на 255. В Fashion-MNIST валидация стратифицирована, а интенсивности дополнительно нормализуются по среднему и стандартному отклонению обучающей выборки. CIFAR-10 и CIFAR-100 [12] содержат RGB-изображения 32×32 с десятью и ста классами. Используется стратифицированное разбиение 45 000, 5000 и 10 000 изображений. Каждый канал нормализуется по статистикам обучения. Валидационные индексы для Fashion-MNIST и CIFAR получены с начальным состоянием генератора 20261027. Аугментации отсутствуют.

Во всех сравнениях ширина скрытых слоёв 160, наклоны BiReLU одинаковы, шапка линейная. Используются AdamW [13], скорость обучения 0,003 и размер пакета 512. Для полного сравнения на MNIST коэффициент убывания весов нулевой. В компактной серии на остальных данных коэффициент 0,01 применяется к скрытым скалярным весам маршрутизирующей модели или матрицам скрытых линейных слоёв Dense. Смещения, факторы логитов и выходная шапка исключены из убывания весов. На MNIST испытаны два и четыре скрытых слоя по 500 эпох на одном начальном состоянии генератора (seed 0). Для компактных сетей и их контролей используются четыре слоя. Число начальных состояний и бюджеты указаны для каждого сравнения. Полные модели сравниваются с моделями, имеющими одинаковую ширину. Факторизованный Dense использует произведение двух линейных матриц заданного ранга в каждом скрытом слое.

Часть исходных компактных запусков CIFAR-10 обучалась на NVIDIA Tesla T4, остальные на Apple Metal. В итоговой серии CIFAR-10 и Fashion-MNIST логиты для расчёта метрик пересчитаны на одном устройстве Metal. Серию CIFAR-100 обучили на Metal после фиксации конфигураций сжатия по CIFAR-10. Кросс-энтропия в таблицах относится к исходным логитам, если явно не указана температурная калибровка.

5.2. Полные модели

Таблица 1: Точность полных моделей на тесте по чекпойнту с максимальной валидационной точностью. Глубина указана по числу скрытых слоёв. Каждая строка соответствует одному запуску с seed 0.

Набор	Слоёв	Параметры		Test accuracy, %	
		Dense	Средние	Dense	Средние
MNIST	2	152 970	153 290	98,00	95,49
MNIST	4	204 490	205 130	98,28	95,99
CIFAR-10	3	544 810	545 290	50,60	50,31
CIFAR-10	4	570 570	571 210	50,58	51,67

В полных моделях Dense обучает полную матрицу весов, а слой средних обучает полную матрицу логитов маршрутизации. На MNIST полный Dense точнее при близком числе параметров (табл. 1). На CIFAR-10 при трёх скрытых слоях тестовые результаты выбранных чекпойнтов близки, а при четырёх модель средних опережает Dense на 1,09 п.п. Обе сети в этой серии продолжены до 2000 эпох. Ровно на 2000-й эпохе четырёхслойные модели дали 51,28% у Dense и 51,08% у модели средних при тестовой кросс-энтропии 4,534 и 1,449 соответственно. Это сравнение на одном seed чувствительно к правилу выбора чекпойнта. Часть траекторий продолжалась на Metal с сохранённым состоянием оптимизатора после исходных

запусков на T4.

5.3. Модели с факторизацией

Таблица 2: Тестовая точность факторизованных моделей. У Dense ранг r относится к линейным весам, у маршрутизации к логитам в (3). Два порядка BiReLU используют одинаковое число параметров. Тире обозначает отсутствие сопоставимого запуска.

Набор	r	Эпох	Запусков	Параметры		Test accuracy, %		
				Dense	Маршрут.	Dense	До суммы	После суммы
CIFAR-10	4	800	1	19 018	19 658	36,19	48,17	—
CIFAR-10	8	800	3	35 786	36 426	44,60	49,12	48,46
CIFAR-10	16	800	1	69 322	69 962	49,47	49,67	—
CIFAR-10	32	800	1	136 394	137 034	51,40	50,84	—
CIFAR-10	8	1200	3	—	36 426	—	50,58	49,63
Fashion-MNIST	8	500	3	17 482	18 122	85,92	85,60	85,94
CIFAR-100	8	400	3	50 276	50 916	17,73	20,63	20,07

В табл. 2 все модели имеют четыре скрытых слоя. Точность усреднена по указанному числу seed, а чекпойнты выбраны по валидации в пределах указанного бюджета. На CIFAR-10 при $r = 8$ и 800 эпохах оба порядка BiReLU опередили факторизованный Dense: на 4,52 п.п. при активации до суммы и на 3,86 п.п. при активации после неё. При бюджете 400 эпох преимущество варианта до суммы над Dense составляло 2,54 п.п. Оценки при $r = 4, 16, 32$ получены на одном seed. В этой серии разрыв сосредоточен при малых рангах. На Fashion-MNIST точность трёх компактных моделей близка.

Компактная модель и полный Dense. Отдельная серия сравнивает компактную модель с полным Dense. На CIFAR-10 при бюджете 1200 эпох получено 50,58% при 36 426 параметрах против 50,81% при 570 570 параметрах. Это уменьшение числа параметров в 15,7 раза. Исследовательский парный 95%-й bootstrap-интервал разницы по изображениям и seed равен $[-1{,}44; +1{,}08]$ п.п. Его нижняя граница ниже порога неуступания в один процентный пункт. На Fashion-MNIST полная модель дала 89,01% против 85,60% у компактной, на CIFAR-100 — 21,71% против 20,63%.

Положение BiReLU. На CIFAR-10 при 1200 эпохах и одинаковой факторизации логитов средняя точность составила 50,58% при активации до суммы и 49,63% после неё. Парный интервал разницы $[-0{,}23; +2{,}21]$ п.п. включает нуль. На Fashion-MNIST соответствующие результаты 85,60% и 85,94%, на CIFAR-100 — 20,63% и 20,07%. Пример из раздела 4 устанавливает преимущество в представимости конкретной функции, а эффект порядка операций в классификации зависит от набора и режима обучения.

На CIFAR-10 исходная тестовая кросс-энтропия составила 1,410 у компактной модели и 4,011 у Dense. После температурной калибровки по валидации [14] значения составили 1,399 и 1,430 соответственно. На Fashion-MNIST после такой калибровки полный Dense дал 0,348 против 0,400 у модели средних. Кросс-энтропия чувствительна к масштабу логитов, а точность зависит от порядка классов.

6. Компиляция обученного слоя

6.1. Явные маршруты и остаточная масса

Пусть S_m — множество K входов с наибольшими p_{mi}, а $G_1, \ldots, G_J$ — фиксированное разбиение входных координат. Для группы G_j зададим множество отброшенных входов $T_{mj} =$

$G_j \setminus S_m$ и его массу $\tau_{mj} = \sum_{i \in T_{mj}} p_{mi}$. Положим $c_{mj} = \tau_{mj}/|T_{mj}|$ при $|T_{mj}| > 0$ и $c_{mj} = 0$ иначе. В Q явные вероятности сохраняются, остальные принимают значение c_{mj} в своей группе. Строки Q нормированы, а сумма вероятностей в каждой группе совпадает с исходной. Обозначим $f_m(t) = g(w_m t + b_m)$ и номер группы входа i через $j(i)$. Тогда

$$h_m^Q(x) = \sum_{j=1}^{J} c_{mj} \sum_{i \in G_j} f_m(x_i) + \sum_{i \in S_m} (p_{mi} - c_{m,j(i)}) f_m(x_i). \tag{5}$$

В первой сумме используются все координаты группы. Вторая сумма восстанавливает явные маршруты, вычитая их вклад в общий хвост. Для экспериментов с изображениями используются либо одна группа, либо пространственные блоки 8×8 в каждом RGB-канале. В Fashion-MNIST соответствующая абляция использует блоки 7×7 одного канала.

Утверждение 1. *Пусть на группе координат вычисляется $\sum_i a_i(x_i)$, а все координаты имеют общий допустимый диапазон значений. Гистограмма значений группы определяет эту сумму для всех входов тогда и только тогда, когда $a_i(t) - a_j(t)$ постоянно по t для любых i, j.*

Доказательство. Перестановка двух значений u, v между позициями i, j сохраняет гистограмму. Следовательно, $a_i(u) + a_j(v) = a_i(v) + a_j(u)$. Перенос двух слагаемых показывает, что разность функций постоянна. Если разности постоянны, выберем одну общую функцию a и представим $a_i = a + d_i$. Тогда сумма равна сумме a по гистограмме плюс $\sum_i d_i$. □

Общая пара w_m, b_m и постоянный коэффициент c_{mj} обеспечивают условие для первого слагаемого (5). Гистограмма группы не зависит от номера выходного нейрона. Отдельные пороги $-b_m/w_m$ меняют только способ чтения статистик. Для фиксированного x сохранение массы в каждой группе также даёт оценку

$$|h_m^P(x) - h_m^Q(x)| \le \frac{1}{2} \sum_{j=1}^{J} \|(p_m - q_m)_{G_j}\|_1 \left(\max_{i \in G_j} f_m(x_i) - \min_{i \in G_j} f_m(x_i) \right). \tag{6}$$

Она получается вычитанием середины диапазона откликов из каждого слагаемого группы. При этом вклад середины равен нулю, поскольку сумма разностей вероятностей равна нулю. Поэтому при одинаковом бюджете важна согласованность групп с пространственной структурой изображения. Оценка (6) относится к одному слою и фиксированному входу.

6.2. Префиксные статистики и реализация

Для каждого объекта накапливаем по группам префиксные числа элементов и суммы значений относительно упорядоченных границ. Для ненулевого w_m активная область $w_m x_i + b_m > 0$ лежит выше порога $-b_m/w_m$ при $w_m > 0$ и ниже при $w_m < 0$. Обозначим число и сумму элементов в этой области через n_+ и s_+. Пусть n и s — число и сумма элементов всей группы. Из определения g получаем

$$\sum_{i \in G_j} f_m(x_i) = \tfrac{1}{2}(w_m s + b_m n) + \tfrac{3}{2}(w_m s_+ + b_m n_+). \tag{7}$$

При $w_m = 0$ сумма равна $n g(b_m)$. При совпадении значения с порогом его вклад в активную часть равен нулю.

При компиляции пороги нейронов фиксируются. Для 8-битного изображения таблица заранее переводит 256 интенсивностей каждого канала в интервалы между порогами после нормализации. Для скрытых вещественных активаций начальный интервал находится по 12 старшим битам ключа, сохраняющего порядок чисел формата FP32. Сравнения с действительными порогами уточняют интервал. Такое уточнение сохраняет исходные значения

активаций и исключает их сортировку на каждом объекте. Накопленные счётчики и суммы дают (7), после чего применяется разреженная поправка (5).

При J группах и не более M порогов на группу накопление статистик первого слоя и оценка выхода занимают $O(N + JM + MK)$ операций на объект после построения таблицы. На скрытых слоях добавляются уточняющие сравнения. Число явных коэффициентов и индексов масштабируется как MK, а групповых коэффициентов — как MJ. При фиксированном размере блока число групп растёт вместе с N. Фактическая задержка зависит от устройства, типов данных, реализации и размера пакета.

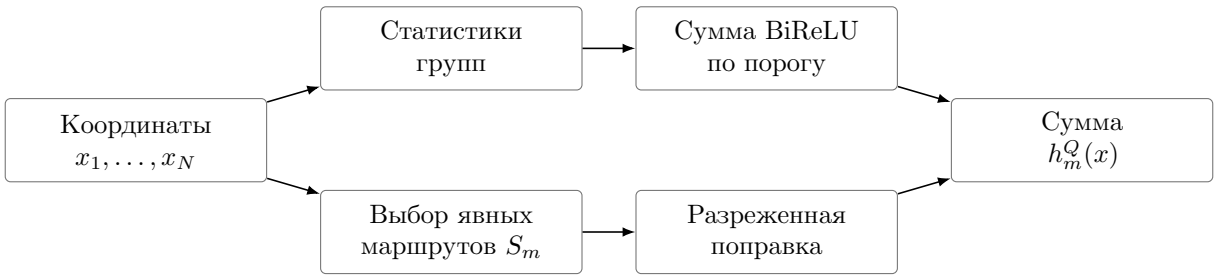


Рис. 1: Вычисление выхода сжатого слоя по (5). Статистики группы вычисляются один раз для объекта. Каждый выход использует свой порог и собственные явные маршруты.

6.3. Подгонка коэффициентов на обучающих данных

Для наиболее компактного испытанного представления используется одна равномерная компонента на слой. Явные маршруты S_m фиксируются по top-K. Положим $a_m(x) = N^{-1} \sum_i f_m(x_i)$ и представим выход как

$$\widetilde{h}_m(x) = a_m(x) + \sum_{i \in S_m} \beta_{mi}\big(f_m(x_i) - a_m(x)\big), \quad \beta_{mi} \geq 0, \quad \sum_{i \in S_m} \beta_{mi} \leq 1. \tag{8}$$

Оставшаяся масса умножает равномерное распределение на всех входах. Параметры w_m, b_m остаются фиксированными. В каждом слое минимизируется средний квадрат отклонения от откликов учителя на 1536 обучающих изображениях со штрафом $\lambda\|\beta_m - \beta_m^{(0)}\|_2^2$. Для матрицы признаков A_m из скобок в (8) и матрицы Грама $G_m = A_m^{\mathsf{T}} A_m/1536$ берётся $\lambda = 10^{-8} + 10^{-4}\,\mathrm{tr}(G_m)/K$. Для следующего слоя входом служат выходы уже сжатых предшествующих слоёв. Используются 160 шагов ускоренного проецируемого градиента. Последняя линейная шапка восстанавливается гребневой регрессией по логитам учителя на тех же изображениях. Оптимизируются веса шапки со штрафом относительно исходных весов и свободное смещение. При центрированных признаках H параметр гребневой регрессии равен $10^{-6} + 0{,}01\,\mathrm{tr}(H^{\mathsf{T}}H)/d_L$.

Экспорт хранит вероятности явных входов в FP16, индексы в `uint16`, массу отброшенных входов, параметры активаций и шапку в FP32. Погрешность нормировки из-за округления вероятностей отделяется от погрешности исполнителя. В замерах на центральном процессоре используются оба формата хранения вероятностей, FP16 и FP32. FP32-вариант использует те же численные значения, что и FP16-экспорт, после преобразования формата.

7. Эксперименты по компиляции

7.1. Сжатие и контроль механизма

Бюджет $K = (64, 32, 64, 32)$ выбран на валидации CIFAR-10 в серии с подбором опоры по активациям. При этом же бюджете оценены все методы из серии. Для переноса использована опора top-K по вероятностям с подгонкой масс. Для Fashion-MNIST и CIFAR-100 переносится

тот же бюджет и порядок подгонки. На каждом наборе параметры подгоняются только на собственном обучающем разбиении. На CIFAR-100 для калибровки взяты 15 изображений каждого класса и ещё 36 изображений из оставшейся обучающей части. Точные учителя имеют те же веса и те же чекпойнты, из которых получены сжатые модели. Время обучения учителя и калибровки в задержку вывода не включается.

Таблица 3: Сжатие уже обученной сети. Точность и кросс-энтропия по формуле (1) усреднены по трём seed на тесте.

Набор	Точный учитель		Сжатая модель		Δ, п.п.
	Accuracy, %	CE	Accuracy, %	CE	
CIFAR-10	50,58	1,4100	50,23	1,4195	−0,35
Fashion-MNIST	85,60	0,4028	85,46	0,4065	−0,14
CIFAR-100	20,63	3,3538	20,36	3,3678	−0,27

Без послойной подгонки коэффициентов, при том же top-K и с той же гребневой регрессией для шапки, точность на CIFAR-10 составила 49,68%. Подгонка увеличила её до 50,23%. При выборе опорных входов по ошибке активаций вместо вероятности и той же подгонке получено 50,06%. В этих экспериментах улучшение даёт подгонка масс, а проверенный способ смены опорного множества качества не добавил. Парные исследовательские интервалы разницы с учителем составляют [−0,75; +0,02] п.п. для CIFAR-10 и [−0,31; +0,03] п.п. для Fashion-MNIST. Интервалы получены парным перевыбором seed и общих тестовых изображений. Три seed ограничивают надёжность этих оценок. На CIFAR-100 бюджет перенесён без подбора на его тесте. Результаты приведены в табл. 3.

Таблица 4: Абляции первого слоя при 64 явных маршрутах. В случайном и пространственном разбиениях одинаковы число и размеры групп. В контроле активации одинаковы также явные маршруты и массы групп.

Контроль	Метрика	Первый вариант	Второй вариант
Пространственные / случайные группы	CIFAR-10, val accuracy	51,15%	49,60%
Пространственные / случайные группы	CIFAR-10, MSE слоя	0,000226	0,001546
Пространственные / случайные группы	CIFAR-100, test accuracy	20,55%	19,09%
Среднее BiReLU / BiReLU среднего	Fashion-MNIST, val accuracy	85,79%	83,43%
Среднее BiReLU / BiReLU среднего	CIFAR-100, test accuracy	20,55%	20,23%

Таблица 4 разделяет эффект выбора групп и эффект нелинейности. В первых строках массы сохраняются по 48 группам, пространственным или случайным внутри цветовых каналов. Средний квадрат ошибки первого слоя вычислен на фиксированных 512 валидационных изображениях CIFAR-10 и усреднён по нейронам, изображениям и трём seed. Для случайного разбиения дополнительно усреднены три фиксированные перестановки. Улучшение соответствует меньшей ошибке приближения в пространственно согласованных группах. В последних строках различается только способ вычисления хвоста. Сохранение отклика по обе стороны порога устраняет ошибку (4) для сжатого распределения, что даёт особенно заметный эффект на Fashion-MNIST.

7.2. Память и задержка полного вывода

Сравнение реализаций проведено на CPU Apple M3 Pro для CIFAR-10, seed 0. Входы представляют нормализованные исходные 8-битные изображения в FP32. Вычисляется весь классификатор с линейной шапкой. Используются прогрев, один поток вычислительной библиотеки и медиана по 11 чередующимся сериям замеров. Число байтов включает постоянные массивы и явный рабочий буфер для пакета из одного изображения. Внешний вход, объекты Python и внутренние выделения памяти библиотек в это число не входят. Для точного кеша размер рабочей памяти оценён по массивам реализации.

Таблица 5: Полный вывод на CPU. Задержка дана за пакет. Для сжатых вариантов формат относится к хранению явных вероятностей.

Исполнитель	Пакет 1, мкс	Пакет 128, мс	Байты при пакете 1
Точный кеш маршрутов, FP32	59,92	6,435	≈2 288 760
Сжатый, префиксы после сортировки, FP32	24,72	2,730	209 172
Сжатый, интервалы порогов, FP32	21,29	2,277	221 460
Сжатый, интервалы порогов, FP16	20,95	2,234	160 020
Dense FP32, Accelerate	13,78	0,198	2 283 600

При хранении явных вероятностей в FP32 предложенный исполнитель быстрее точного кеша исходной модели в 2,81 раза при пакете из одного изображения и использует в 10,3 раза меньше байтов в учтённых массивах (табл. 5). При сравнении двух реализаций одного сжатого распределения замена сортировки интервалами сокращает задержку с 24,72 до 21,29 мкс ценой дополнительной таблицы (рис. 2). Для этой конфигурации на всех 10 000 тестовых изображениях seed 0 сохранён класс, предсказанный формулой сжатого слоя в PyTorch. Максимальное абсолютное расхождение логитов для интервалов составляет $1{,}51 \cdot 10^{-5}$ на нормализованном входе и $2{,}60 \cdot 10^{-5}$ на исходном uint8-входе. Это проверка численной эквивалентности вычисления Q.

Сериализованное представление CIFAR-10 с FP16-вероятностями занимает 137 000 байт на модель и 4459 байт внешних метаданных. Оно отличается от резидентного представления исполнителя, в которое входят пороги и таблицы. Исходные обучаемые FP32-параметры занимают 145 704 байта. Основное сокращение объёма в таблице относится к полному кешу вероятностей, возникающему при развёртывании компактных логитов.



Рис. 2: Медианная задержка на одном изображении CIFAR-10, seed 0, CPU Apple M3 Pro. Интервалы порогов сокращают время относительно сортировки при том же сжатом представлении. Оптимизированный Dense использует более быстрые линейные операции.

7.3. Сравнение при близком бюджете хранения

Отдельно обучен факторизованный Dense с рангом, выбранным по объёму INT8-весов, 31 для CIFAR-10 и 41 для CIFAR-100. Квантование выполняется после обучения с построчными коэффициентами масштаба. Контроль обучается 400 эпох на трёх seed. У маршрутизирую-

щего учителя бюджет 1200 эпох на CIFAR-10 и 400 эпох на CIFAR-100. В табл. 6 приведена точность после декодирования весов и размер формата. Этот контроль проверяет качество при близком бюджете хранения. Задержка низкорангового INT8-контроля здесь не измерялась.

Таблица 6: Точность при близком объёме представления. Для сжатого CIFAR-10 указан бинарный объём без внешних метаданных. Остальные размеры рассчитаны по формату хранения.

Набор	Сжатая маршрутизация		Factor Dense INT8	
	Байты	Test, %	Байты	Test, %
CIFAR-10	137 000	50,23	137 248	50,91
CIFAR-100	194 960	20,36	194 448	23,17

При близком объёме INT8 Factor Dense точнее на обоих наборах. Подгонка его шапки на тех же 1536 изображениях даёт 50,88% и 23,13%. Оптимизированный FP32 Dense из табл. 5 быстрее сжатой маршрутизации при обоих размерах пакета. Наблюдаемое преимущество относится к исполнению конкретной обученной модели, когда требуется сохранить её прогнозы и уменьшить объём кеша.

8. Обсуждение и ограничения

Архитектурное сравнение использует небольшие классификаторы на развёрнутых изображениях. Оно характеризует исследуемый порядок агрегации в контролируемом семействе моделей. Перенос выводов на свёрточные и другие пространственные архитектуры требует отдельного сравнения. При трёх seed точность оценок разброса ограничена, а эксперименты MNIST с одним seed дают описание отдельного запуска. CIFAR-10 и Fashion-MNIST использовались при разработке. CIFAR-100 проверяет перенос фиксированного бюджета сжатия на новый набор, но также состоит из 8-битных изображений небольшого размера.

Обобщение подхода на произвольные кусочно-линейные функции возможно через несколько пороговых запросов. Стоимость и полезность такой реализации зависят от числа порогов и общности функций на рёбрах. Динамическая маршрутизация с весами, зависящими от объекта, потребует отдельно вычислять массы групп. Прототип с интервальной оценкой ошибки даёт строгие границы для заданных условий, но на валидационной подвыборке CIFAR-10 при $K = 96$ смог сертифицировать лишь 0,67% примеров при фактическом совпадении предсказаний 96,0%. Поэтому результаты измеренного ускорения не используют сертификат.

9. Заключение

Исследован слой статической маршрутизации, вычисляющий среднее нелинейных откликов с общими параметрами активации на выход. Компактный вариант на CIFAR-10 достиг близкой к полносвязному контролю точности при меньшем числе обучаемых параметров. На MNIST и Fashion-MNIST измерен разрыв в пользу полного Dense. Формальный пример показывает, какие свойства входных значений может сохранить нелинейность до сжимающей агрегации.

Предложенный компилятор вычисляет хвост сжатого статического распределения через общие префиксные статистики и индивидуальные пороги. После калибровки потеря точности на трёх наборах составила от 0,14 до 0,35 п.п. относительно исходных маршрутизирующих моделей. На CPU исполнитель уменьшил размер явной памяти и задержку по сравнению с точным кешем этой же сети. Практический сценарий применения — развёртывание уже обученного статического слоя с ограничением на объём кеша. Дальнейшая проверка должна установить, как изменяется выигрыш на других процессорах и для слоёв с несколькими

порогами.

Воспроизводимость. Архив экспериментов `compact_routing_article/reproducibility.zip` содержит реализации, протоколы, чекпойнты, истории обучения, результаты абляций и CPU-замеров. Источники наборов указаны в литературе. В сопроводительных материалах к исходнику приведена карта таблиц и файлов с метриками.

Список литературы

[1] Y. Tay и др. ``Synthesizer: Rethinking Self-Attention for Transformer Models''. В: *Proceedings of the 38th International Conference on Machine Learning*. Т. 139. PMLR. 2021, с. 10183—10192. URL: https://proceedings.mlr.press/v139/tay21a.html.

[2] Z. Liu и др. ``KAN: Kolmogorov-Arnold Networks''. В: *International Conference on Learning Representations*. 2025. URL: https://arxiv.org/abs/2404.19756.

[3] A. Bayeh, S. Sadaoui и M. Mouhoub. *TruKAN: Towards More Efficient Kolmogorov-Arnold Networks Using Truncated Power Functions*. arXiv:2602.03879. 2026. URL: https://arxiv.org/abs/2602.03879.

[4] L. Ribar и др. ``SparQ Attention: Bandwidth-Efficient LLM Inference''. В: *Proceedings of the 41st International Conference on Machine Learning*. Т. 235. PMLR. 2024, с. 42558—42583. URL: https://proceedings.mlr.press/v235/ribar24a.html.

[5] H. Yu и др. *OpenAttention: One Percent Is Enough with Tail Proxies*. arXiv:2610.30951. 2026. URL: https://arxiv.org/abs/2610.30951.

[6] F. Porikli. ``Constant Time O(1) Bilateral Filtering''. В: *IEEE Conference on Computer Vision and Pattern Recognition*. 2008. DOI: 10.1109/CVPR.2008.4587843.

[7] N. Langrené и X. Warin. *Fast and Stable Multivariate Kernel Density Estimation by Fast Sum Updating*. arXiv:1712.00993, version 2. 2018. URL: https://arxiv.org/abs/1712.00993.

[8] E. Frantar и D. Alistarh. ``SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot''. В: *Proceedings of the 40th International Conference on Machine Learning*. Т. 202. PMLR. 2023, с. 10323—10337. URL: https://proceedings.mlr.press/v202/frantar23a.html.

[9] T. Dao, A. Gu, M. Eichhorn, A. Rudra и C. Ré. ``Learning Fast Algorithms for Linear Transforms Using Butterfly Factorizations''. В: *Proceedings of the 36th International Conference on Machine Learning*. Т. 97. PMLR. 2019, с. 1517—1527. URL: https://proceedings.mlr.press/v97/dao19a.html.

[10] Y. LeCun, L. Bottou, Y. Bengio и P. Haffner. ``Gradient-Based Learning Applied to Document Recognition''. В: *Proceedings of the IEEE* 86.11 (1998), с. 2278—2324. DOI: 10.1109/5.726791.

[11] H. Xiao, K. Rasul и R. Vollgraf. *Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv:1708.07747. 2017. URL: https://arxiv.org/abs/1708.07747.

[12] A. Krizhevsky, V. Nair и G. Hinton. *CIFAR-10 and CIFAR-100 Datasets*. URL: https://www.cs.toronto.edu/~kriz/cifar.html.

[13] I. Loshchilov и F. Hutter. ``Decoupled Weight Decay Regularization''. В: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=Bkg6RiCqY7.

[14] C. Guo, G. Pleiss, Y. Sun и K. Q. Weinberger. ``On Calibration of Modern Neural Networks''. В: *Proceedings of the 34th International Conference on Machine Learning*. Т. 70. PMLR. 2017, с. 1321—1330. URL: https://proceedings.mlr.press/v70/guo17a.html.

Compilation of Static Routing with Nonlinearity Before Aggregation

Abstract. We study a multilayer classifier whose hidden units take a weighted average of scalar nonlinear responses. Each unit shares one scale and one bias across input coordinates and learns a static routing distribution. On CIFAR-10, factorized routing logits reduce the parameter count from 570,570 to 36,426 and yield 50.58% test accuracy compared with 50.81% for the full dense control. Dense controls achieve higher accuracy on MNIST and Fashion-MNIST in the corresponding comparisons. At deployment, materializing the routing probabilities can largely offset the storage benefit of factorized logits. We compile a trained layer into explicit routes and group-wise tail masses. A common prefix representation evaluates the piecewise-linear tail response at each neuron's own threshold. Calibration on 1,536 training images yields losses of 0.35, 0.14, and 0.27 percentage points relative to the exact routing teachers on CIFAR-10, Fashion-MNIST, and CIFAR-100, respectively. On a single CPU thread of an Apple M3 Pro, with batch size one and 32-bit probability storage, the compiled classifier runs 2.81 times faster than exact cached routing and uses 10.3 times less memory for model arrays and explicit workspace. The optimized dense control remains faster. The results identify a deployment benefit for a fixed trained routing model.

Keywords. Static routing, multilayer perceptron, piecewise-linear activation, neural network compression, prefix sums, inference latency.